

# CanSat Comms Library Design

Mubashir Osmani

## 1. Running on the microcontroller

The flight computer is a small Cortex-M, so the main thing I wanted to avoid was dynamic memory. Nothing in the library calls `new` or `malloc` or uses `std::vector`, and it compiles with exceptions and RTTI turned off. Instead, every function takes a buffer from the caller along with its size, and returns a status code saying whether it worked. The biggest frame we can build is 118 bytes (with the payload capped at 100 bytes) and the biggest message is 7 bytes. Both are constants in the headers, so the buffers can just be static arrays and we know exactly how much RAM the radio code uses before we fly. That means there's no chance of the heap fragmenting or an allocation failing halfway through the mission.

I also tried not to copy data around more than necessary. When a 0x90 frame is parsed, the result just points at the payload inside the receive buffer instead of copying it out. Everything is a single loop over the input, so it takes about the same time every time. The library doesn't touch the UART at all, which is also what let me test all of it on my laptop with GoogleTest.

The downside is that the payload limit is fixed at compile time, so the buffers are always sized for the worst case even though our messages are tiny. The parsed payload is only valid until the receive buffer gets reused, so commands have to be decoded right away. It's also a bit clunkier to use than containers since you have to pass sizes around and check every return value. For flight code I think that's worth it.

## 2. Payload format

Each payload starts with one byte saying what kind of message it is, followed by the struct fields in order:

| ID   | Message          | Bytes                                                     |
|------|------------------|-----------------------------------------------------------|
| 0x01 | CommandMechanism | ID, mechanism_id, value (3 total)                         |
| 0x02 | CommandTelemetry | ID, is_on (2 total)                                       |
| 0x03 | OutTelemetry     | ID, seconds (4 bytes), deployed flags (2 bytes) (7 total) |

I kept every field the same size as in the struct so nothing gets cut off. The `bool` is sent as a full byte and has to be 0 or 1. Anything bigger than a byte is big-endian, mostly because XBee already uses big-endian for its own length and address fields, so the whole frame reads the same way. I write each byte with shifts instead of `memcpy`-ing the struct. That way compiler padding and the CPU's byte order don't change what goes over the radio, and the CanSat and the ground station laptop produce the same bytes.

The ID byte is what lets the receiver tell a mechanism command apart from a telemetry command. Each ID also has one fixed length, which makes it easy to check that a message isn't cut short or padded with junk. I didn't use 0x00 as an ID so a payload of all zeros doesn't decode as something real. I didn't put a checksum in the payload because the XBee frame already has one.

## 3. Bad or incomplete data

The radio link is going to be noisy, so I assumed anything could show up on the UART. The parser checks the start byte first, then the length field, then waits until the whole frame has arrived, then checks the checksum, and only then looks at what type of frame it is. It never reads a byte before it

knows that byte is actually there. Checking the length early matters because a corrupted length could otherwise make it wait for hundreds of bytes that are never coming.

I check the frame type after the checksum because the XBee sends its own frames too, like 0x8B transmit status. Those aren't errors, so the parser reports them as a different frame type and tells the caller how long they are so they can be skipped. After that, the payload decoder rejects empty payloads, unknown IDs, wrong lengths and an is\_on byte that isn't 0 or 1. Nothing gets written to the output unless the whole message is valid.

For testing I cut a valid frame off at every possible length, flipped every bit one at a time, and fed in a few thousand random buffers. None of those crashed or got accepted as valid.

On the flight software side, if a frame is incomplete it should just keep the bytes and wait for more, with a timeout so it doesn't wait forever. If the start byte, length or checksum is bad, it should drop a byte and look for the next 0x7E. If the frame is fine but it's not one we care about, or the payload doesn't decode, skip it. The most important rule is to never act on a command we're not sure about.

Accidentally deploying a mechanism is a lot worse than missing a command, since the ground station can always resend it. It would also be useful to count each type of error and send the counts down in telemetry, so we can tell whether problems are coming from the radio link or from a bug.